

Types of SDLC



SDLC

SDLC (Software Development Life Cycle) models are structured frameworks guiding software development, with common types including Waterfall (sequential, fixed requirements), Agile (iterative, flexible, customer-focused), Spiral (risk-driven, complex projects), V-Model (testing-focused, validation/verification), Iterative/Incremental (delivers in pieces), and Prototyping (builds quick models for unclear needs). Each model offers different approaches to planning, designing, building, testing, and maintaining software, suiting various project complexities and goals, from small, fixed projects to large, evolving ones.

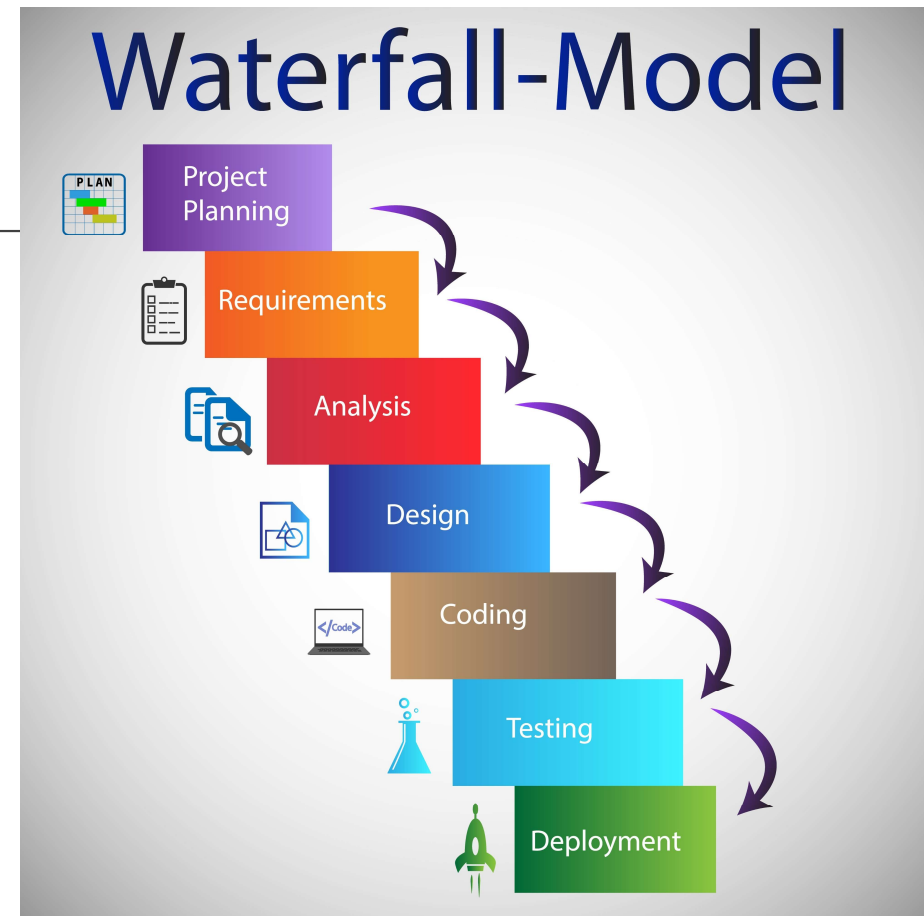


Types of SLDLC

1. Waterfall
2. Agile
3. Spiral
4. Prototype
5. Incremental
6. RAD
7. V-Model
8. DevOps
9. Big-Bang

Waterfall Model

The Waterfall Method is a linear project management approach where a project is completed in distinct stages that move step-by-step in one direction—like a waterfall cascading down rocks. In this model, you cannot proceed to the next phase until the previous one is 100% complete and signed off. It is the traditional opposite of "Agile" methods, which are iterative and flexible.



Waterfall Model for a School LMS

1. Requirements Analysis

The school wants a simple LMS to support online learning. After meeting with teachers, students, and the admin, the following requirements are identified:

Students can log in and view subjects

Teachers can upload lessons and assignments

Students can submit assignments online

Admin can manage users (add/remove teachers and students)

All requirements are clearly documented and finalized before development starts.

Waterfall Model for a School LMS

2. System Design

Based on the requirements, the system design is created:

Database design: tables for users, subjects, lessons, and assignments

User interface design: login page, student dashboard, teacher dashboard

System architecture: web-based LMS accessible through a browser

This phase serves as the blueprint of the LMS.

Waterfall Model for a School LMS

3. Implementation (Coding)

Programmers start coding the LMS based on the approved design:

Login and authentication module

Teacher module for uploading lessons and assignments

Student module for viewing lessons and submitting assignments

Each feature is developed according to the design documents.

Waterfall Model for a School LMS

4. Testing

After coding is completed, the LMS is tested:

Check if students can log in and access their subjects

Verify teachers can upload files correctly

Ensure assignments submitted by students are saved properly

Errors and bugs found during testing are fixed before moving forward.

Waterfall Model for a School LMS

5. Deployment

Once testing is successful, the LMS is deployed:

Installed on the school server

Accounts are given to teachers, students, and admin

The LMS is officially used for classes and assignments

Waterfall Model for a School LMS

6. Maintenance

After deployment, ongoing support is provided:

Fixing bugs reported by users

Updating the system (e.g., adding quiz features)

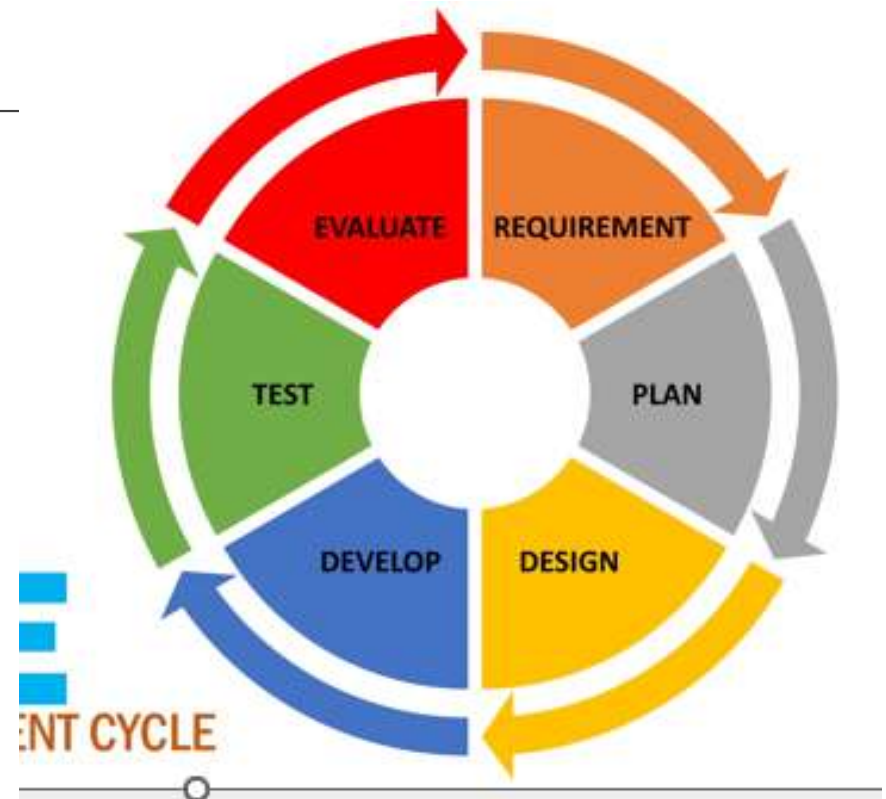
Improving performance and security

This phase continues as long as the LMS is in use.

Agile Model

The Agile Model is an iterative and flexible project management approach that focuses on delivering value to customers in small, consumable increments.

Unlike Waterfall, which tries to plan everything perfectly upfront, Agile assumes that requirements *will* change. It encourages constant feedback, adaptation, and collaboration throughout the project lifecycle.



Agile Model for a School LMS

1. Planning (Sprint Planning)

The development team meets with teachers, students, and the school admin to identify **priority features**. Instead of listing everything at once, they create a **product backlog**, such as:

User login

Subject list for students

Lesson upload for teachers

Features are planned in **short development cycles called sprints** (usually 2–3 weeks).

Agile Model for a School LMS

2. Sprint 1: Basic LMS Functions

In the first sprint, the team develops:

User login and logout

Student dashboard to view enrolled subjects

At the end of the sprint, a **working version** of the LMS is shown to teachers and students for feedback.

Agile Model for a School LMS

3. Sprint 2: Learning Materials

Based on feedback, the team adds new features:

Teacher module for uploading lessons

Student module for viewing and downloading lessons

Users test the features and suggest improvements, such as easier navigation.

Agile Model for a School LMS

4. Sprint 3: Assignments and Submission

The next sprint focuses on:

Creating and uploading assignments

Online assignment submission for students

Viewing submitted work for teachers

Feedback is again collected to refine the system.

Agile Model for a School LMS

5. Testing (Continuous)

Testing happens **during every sprint**, not just at the end:

Each feature is tested as soon as it is developed

Bugs are fixed immediately before moving to the next sprint

Agile Model for a School LMS

6. Release and Deployment

After several sprints, the LMS is released for school-wide use:

Teachers and students start using the system

New features can still be added in future sprints

Agile Model for a School LMS

7. Maintenance and Improvement

The LMS continuously improves:

Adding quizzes, grades, or announcements

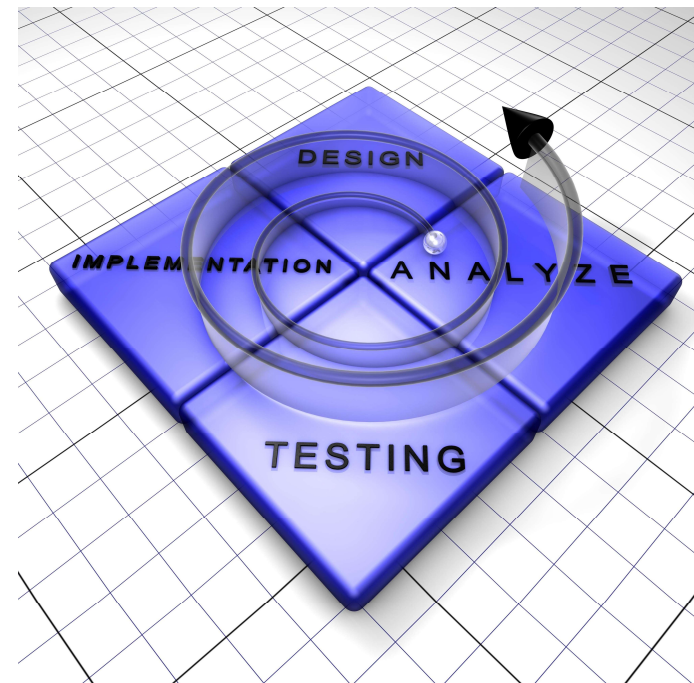
Enhancing security and performance

Adjusting features based on user feedback

Spiral Model

The **Spiral Model** is a risk-driven software development process model. Based on the unique risk patterns of a given project, the Spiral model guides a team to adopt elements of one or more process models, such as incremental, waterfall, or evolutionary prototyping.

It combines the fixed structure of **Waterfall** with the iterative nature of **Agile**, but its primary focus is on **Risk Analysis**.



Spiral Model for a School LMS

Spiral 1: Planning & Risk Identification

The school plans to build a basic LMS.

Objectives: Allow students and teachers to log in and view subjects

Risks Identified:

- Users may find the system hard to use
- Possible login security issues

Solution: Design a simple interface and basic authentication

Development & Testing:

Create login module and simple dashboard

Test login functionality with sample users

Spiral Model for a School LMS

Spiral 2: Content Management

After initial testing, the system moves to the next spiral.

Objectives: Enable teachers to upload lessons and students to view them

Risks Identified:

- Large file uploads may fail
- Students may not access materials properly

Solution: Set file size limits and access control

Development & Testing:

Develop lesson upload and viewing features

Test file uploads and downloads

Spiral Model for a School LMS

Spiral 3: Assignments and Submissions

The system expands with more features.

Objectives: Online assignment creation and submission

Risks Identified:

- Lost student submissions
- Late submission handling issues

Solution: Automatic timestamps and confirmation messages

Development & Testing:

Build assignment and submission modules

Test submission accuracy and deadlines

Spiral Model for a School LMS

Spiral 4: Deployment & Enhancement

The LMS is nearly complete.

Objectives: Deploy LMS for full school use

Risks Identified:

- System performance issues with many users
- Data security concerns

Solution: Load testing and data backup systems

Development & Testing:

Optimize performance

Final testing before deployment

Spiral Model for a School LMS

Maintenance (Ongoing Spirals)

Even after deployment, new spirals continue:

Adding quizzes, grading, and announcements

Addressing new risks and user feedback

Continuous improvement

Prototype Model

The **Prototype Model** is a software development method where a preliminary version of the product (a prototype) is built, tested, and refined before the final system is developed.

It is based on the idea that customers often don't know exactly what they want until they see it. Instead of planning everything on paper (like Waterfall), you build a "rough draft" to show the client immediately.

Prototype Model for a School LMS

. Requirement Gathering (Initial)

The school wants an LMS but is not yet sure about all the features. Teachers and students explain their basic needs:

Easy login

Viewing subjects

Uploading and downloading lessons

The requirements are **not final** at this stage.

Prototype Model for a School LMS

Quick Design

The developers create a **simple design**:

Basic login screen

Sample student and teacher dashboard

Simple menu layout

This design focuses on **how the system looks and works**, not full functionality.

Prototype Model for a School LMS

Build Prototype

A **basic prototype LMS** is developed:

Clickable login page (may not have real security yet)

Sample subject list

Dummy lesson upload and view buttons

The prototype may not be fully functional—it is mainly for demonstration.

Prototype Model for a School LMS

Refine Prototype

Based on feedback:

Improve navigation and layout

Add assignment section in the prototype

Adjust features according to user suggestions

This cycle may repeat **several times** until users are satisfied.

Prototype Model for a School LMS

Develop Final System

Once requirements are clear:

The prototype is used as a guide

The actual LMS is fully coded

Proper security, database, and features are implemented

Prototype Model for a School LMS

Testing, Deployment, and Maintenance

The final LMS is tested thoroughly

Deployed for school-wide use

Maintained and updated as needed

Prototype Model for a School LMS

User Evaluation

Teachers and students try the prototype and give feedback:

“The dashboard is confusing”

“We want a clearer assignment section”

“Buttons should be bigger for mobile use”

This feedback helps clarify what users really want.

Incremental

The **Incremental Model** is a software development approach where the product is built, tested, and delivered in small, manageable pieces (increments).

Instead of delivering the entire system at once (like Waterfall), or constantly changing the plan (like Agile), the **Incremental Model** usually starts with a fixed overall plan, but delivers the product part-by-part.

Think of it like building a house room-by-room. You build the kitchen so you can cook, then you build the bedroom so you can sleep, and finally, you build the garage. You can use the kitchen while the bedroom is being built.

Incremental

Increment 1: User Login and Basic Dashboard

The first working version of the LMS is developed.

Students and teachers can log in

Students can see a list of their subjects

Teachers can see assigned classes

This version is released for limited use and basic testing.

Incremental

Increment 2: Learning Materials

After the first increment is successful, new features are added:

Teachers can upload lessons and files

Students can view and download learning materials

The LMS now supports basic teaching and learning activities.

Incremental

Increment 3: Assignments and Submissions

The system is expanded further:

Teachers can create assignments

Students can submit assignments online

Submission deadlines are recorded

This increment improves assessment and task management.

Incremental

Increment 4: Grading and Feedback

More features are added:

Teachers can grade assignments

Students can view grades and feedback

Basic progress tracking is available

Incremental

Increment 5: System Enhancement

The final increment focuses on improvements:

Better user interface

Improved security and performance

Bug fixes and minor feature updates

Incremental

Testing and Maintenance

Each increment is tested before release

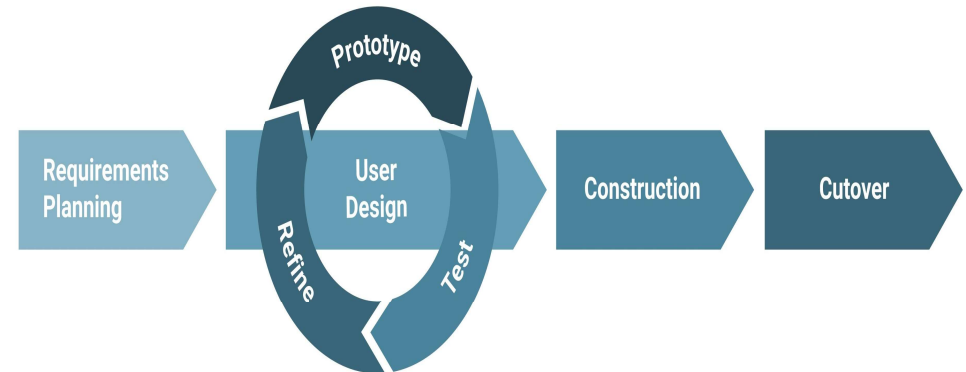
Feedback is collected after every release

The LMS continues to improve even after full deployment

RAPID APPLICATION DEVELOPMENT

Rapid Application Development (RAD) is a development lifecycle designed to give much faster results than the traditional Waterfall method. It treats software like clay rather than steel—it is molded, shaped, and changed constantly based on user feedback.

RAD prioritizes **rapid prototyping** and **iterative delivery** over heavy planning. It compresses the analysis, design, build, and test phases into a series of short, iterative development cycles.



Rapid Application Development (RAD)

Rapid

Requirements Planning

The development team meets with teachers, students, and the school admin in short workshops.

Identify core features: login, subjects, lesson upload, assignments

Agree on scope and timeline

Requirements are defined quickly, not in extreme detail

Rapid

User Design

Users are actively involved in designing the LMS.

Developers create screen mockups of dashboards

Teachers suggest where upload buttons should be placed

Students give feedback on navigation and layout

Design and feedback happen **simultaneously**.

Rapid

Rapid Construction

The LMS is built quickly using:

Reusable modules (login, file upload)

Ready-made frameworks and tools

Continuous testing during development

Features developed rapidly:

Login system

Teacher lesson upload

Student lesson viewing

Assignment submission

Rapid

Cutover (Deployment)

Once the LMS is ready:

Final testing is done

Data is migrated (student and teacher accounts)

System is deployed for actual school use

Users receive quick orientation

Rapid

Ongoing Improvement

Even after deployment:

Feedback is collected

Enhancements (quizzes, grading, announcements) are added

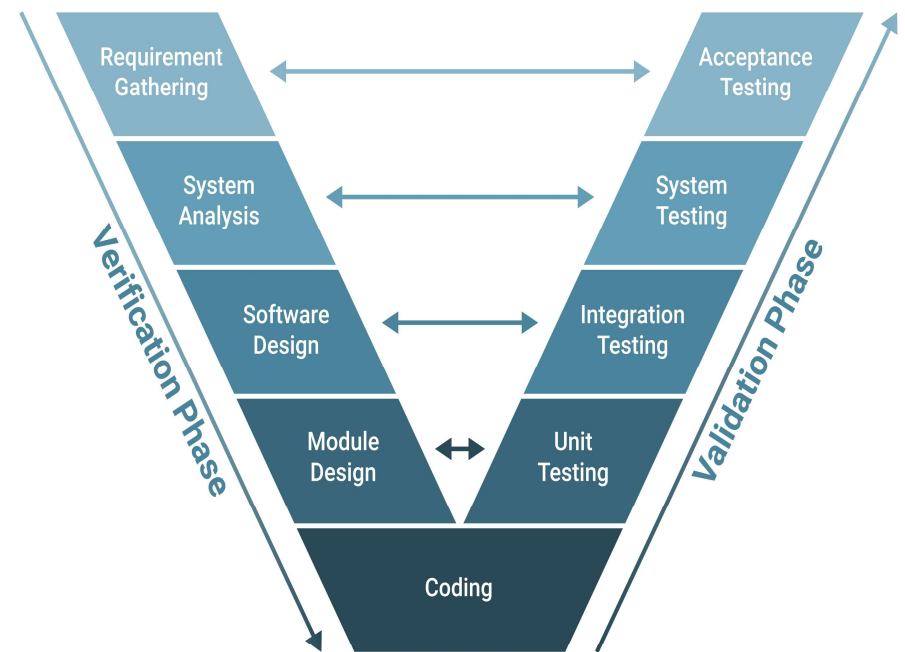
Bugs are fixed quickly

V-Model

The **V-Model** (Verification and Validation Model) is a highly disciplined extension of the Waterfall model.

Instead of moving down in a straight line, the process bends upwards after coding, forming a "V" shape. Its key principle is that **for every single phase of development (on the left), there is a directly corresponding phase of testing (on the right).**

This model ensures that you don't just wait until the end to think about testing; you plan the specific tests *while* you are designing the system.



V Model

Requirements Analysis ↔ Acceptance Testing

Development:

The school defines requirements:

- Students can log in
- Teachers can upload lessons and assignments
- Students can submit assignments and view grades

Testing (Acceptance Test):

Teachers and students test the LMS to confirm it meets all stated requirements.

V Model

System Design ↔ System Testing

Development:

Overall LMS design is created:

- System architecture
- Database structure
- User roles (admin, teacher, student)

Testing (System Test):

Test the entire LMS as a whole:

- Login, lesson upload, assignment submission, grading
- Check performance and security

V Model

High-Level Design ↔ Integration Testing

Development:

Design how modules interact:

- Login module connects to user database
- Assignment module connects to grading module

Testing (Integration Test):

Test interaction between modules:

- Login → Dashboard
- Assignment submission → Teacher grading view

V Model

Coding (Implementation)

Developers code all LMS modules based on the designs.

Unit testing begins as soon as coding starts.

V Model

Deployment and Maintenance

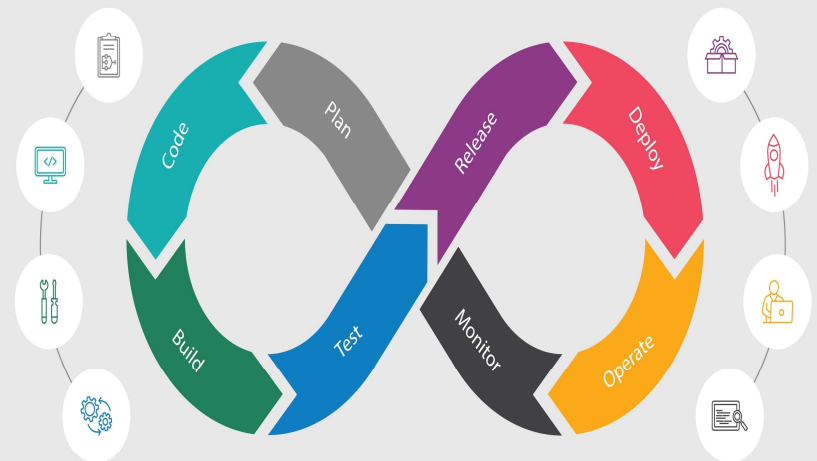
The LMS is deployed to the school server.

Bug fixes, updates, and feature enhancements are done during maintenance.

Dev-Ops

DevOps is not just a methodology; it is a cultural shift and a set of practices that combines **Software Development (Dev)** and **IT Operations (Ops)**.

In traditional models (like Waterfall), "Dev" and "Ops" were enemies. Developers wanted to change things quickly (add features), while Operations wanted to keep things stable (change nothing so the server doesn't crash). DevOps bridges this gap by unifying them into a single team that shares responsibility for the product's entire lifecycle.



DevOps Process

Dev-Ops

Planning

The school, teachers, and developers plan LMS features together:

Online login for students and teachers

Lesson uploads and downloads

Assignment submission and grading

Tasks are listed in a backlog and prioritized.

Dev-Ops

Development (Code)

Developers start coding LMS features:

Login and authentication module

Teacher lesson upload

Student lesson viewing

Code is stored in a shared repository (e.g., Git).

Dev-Ops

Continuous Integration (CI)

Every time code is updated:

The system automatically builds the LMS

Automated tests check if new code breaks existing features

This helps catch errors early.

Dev-Ops

Continuous Testing

Testing happens continuously:

Login testing

File upload testing

Assignment submission testing

Bugs are fixed immediately before deployment.

Dev-Ops

Continuous Deployment (CD)

Once tests pass:

The LMS is automatically deployed to the school server

Updates are released without long downtime

Teachers and students get improvements quickly.

Dev-Ops

Continuous Deployment (CD)

Once tests pass:

The LMS is automatically deployed to the school server

Updates are released without long downtime

Teachers and students get improvements quickly.

Dev-Ops

Monitoring

After deployment:

System performance is monitored

Errors, slow loading, or crashes are detected

User activity is analyzed

Dev Ops

Feedback and Improvement

Feedback from teachers and students is collected:

Requests for quizzes, announcements, or mobile support

Issues are added to the backlog

The cycle repeats for continuous improvement.

Big Bang

The **Big Bang Model** is the simplest—and riskiest—model of software development. It essentially means: "Just start coding."

There is almost no planning, no formal requirements, and no structure. The project is driven by the sheer energy and effort of the developers. You throw time, money, and code at a problem and hope a software product comes out the other side.

Big Bang

Minimal Planning

The school wants a basic LMS but does not prepare detailed documentation.

General idea: an online platform for lessons and assignments

No clear design or detailed requirements

Developers rely on experience and assumptions

Big Bang

. Development (Coding)

Developers start coding all LMS features at the same time:

Login system

Teacher lesson upload

Student lesson viewing

Assignment submission

There is no clear sequence or structured phases.

Big Bang

Testing

After most coding is completed:

The entire LMS is tested at once

Many bugs and errors are discovered

Fixing issues may take more time because components were built together

Big Bang

Deployment

Once major issues are fixed:

The LMS is deployed for school use

Teachers and students start using the system

Big Bang

Maintenance

Bug fixing continues after deployment

Adding new features is difficult and risky

Changes may affect the entire system

Limitations of Big Bang Model

High risk of failure

Difficult to manage changes

Not suitable for large or complex systems like a full LMS